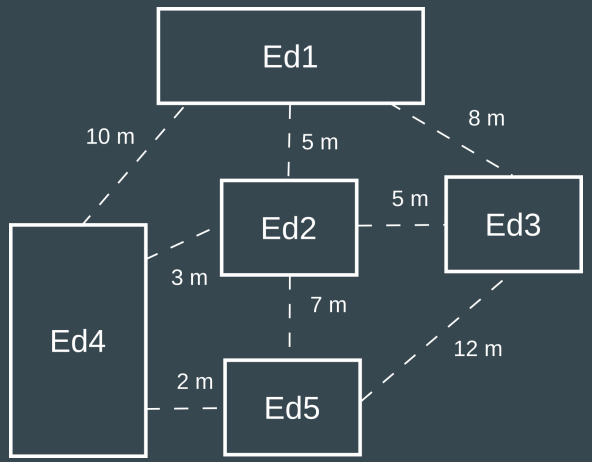
Árbol recubridor mínimo

...

Algoritmos y Estructuras de Datos

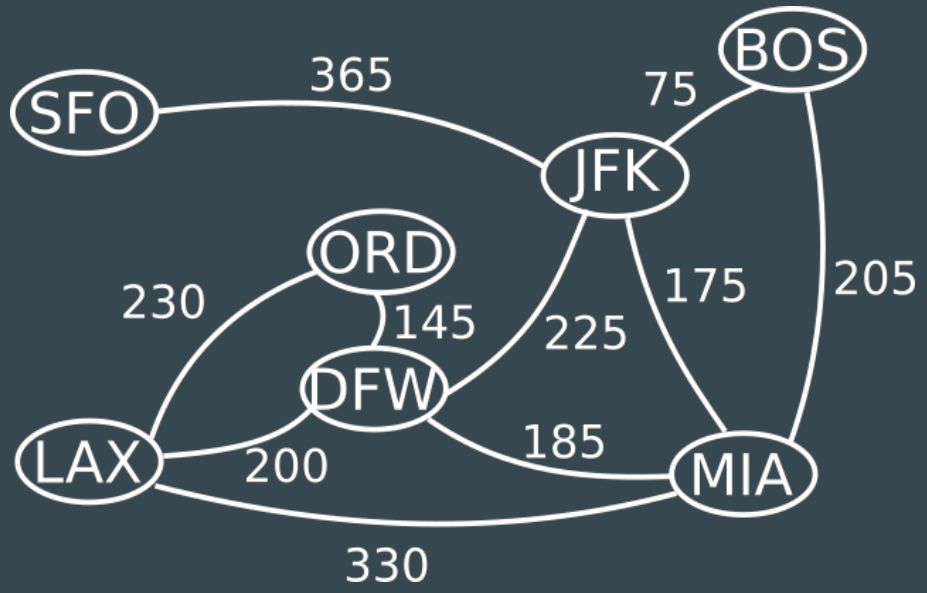
UNRaf UNIVERSIDAD NACIONAL DE RAFAELA

Se desean construir pasarelas entre los edificios de un campus, asegurando que todos los edificios están unidos ¿Qué tendría que tener en cuenta para lograr la combinación de pasarelas más eficiente?



Árbol recubridor mínimo

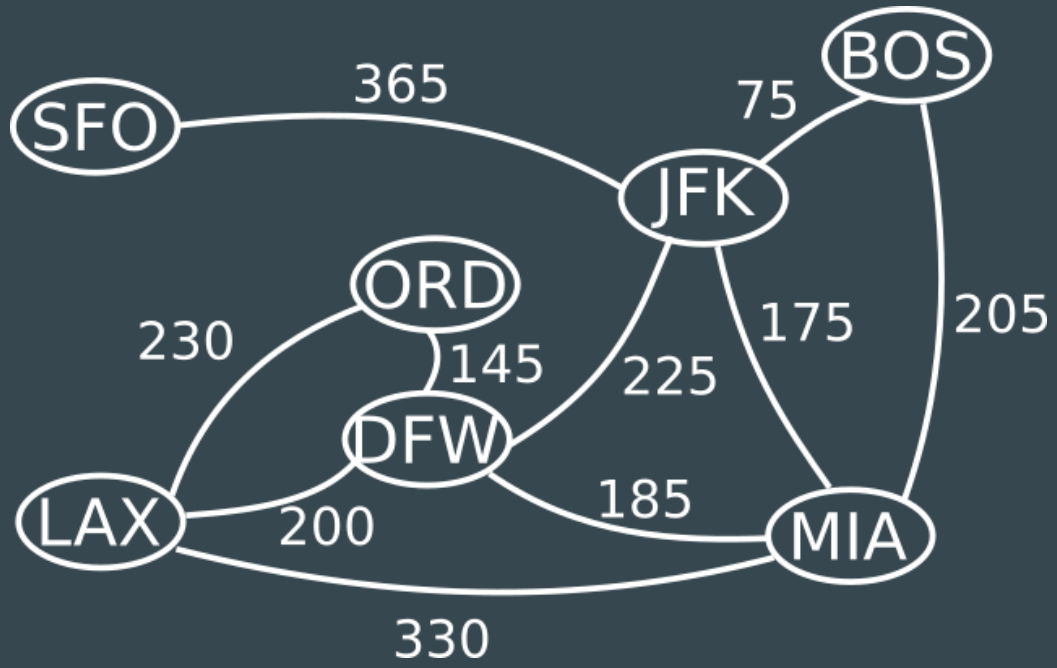
- Un **árbol recubridor** contiene **todos los nodos** de un grafo **no dirigido** y **conectado**.
- Si el grafo posee ***n*** nodos, el árbol recubridor posee ***n*-1** aristas.
- El **árbol recubridor mínimo** es el árbol con **menor peso total**.



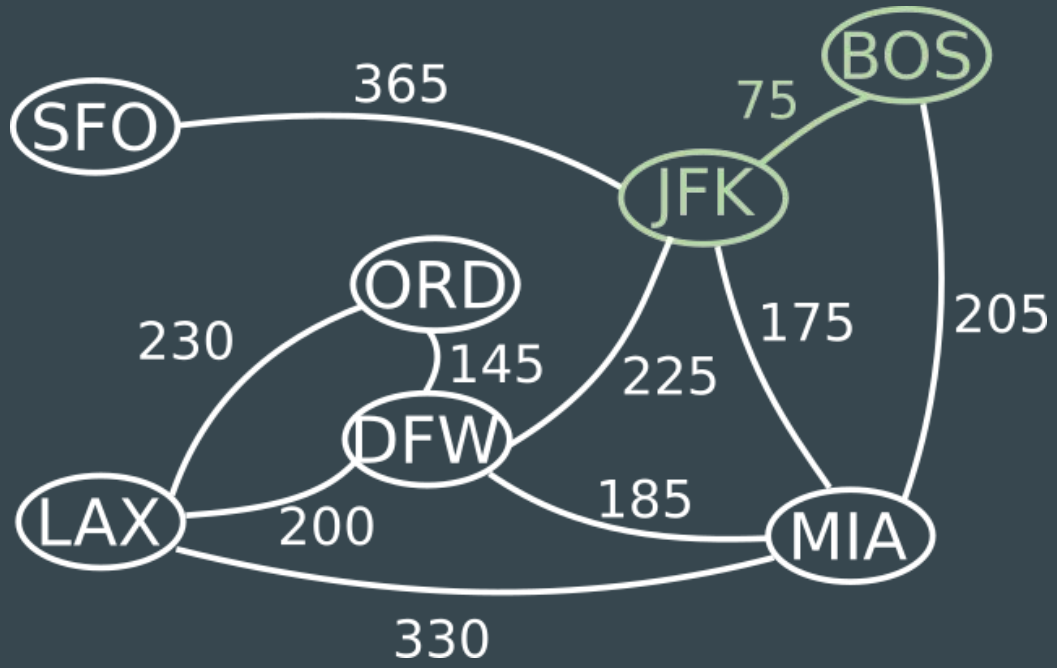
Algoritmo de Kruskal

- Es un algoritmo **ávido**.
- La **heurística** en la que se basa es seleccionar el arco de menor peso,
- A medida que se seleccionan nodos, se van formando distintos subárboles, que serán combinados en un **árbol final**.
- Si la arista de menor peso conecta nodos ya presentes en un mismo árbol, se descarta
- El algoritmo termina cuando se forma un solo árbol de n-1 aristas.

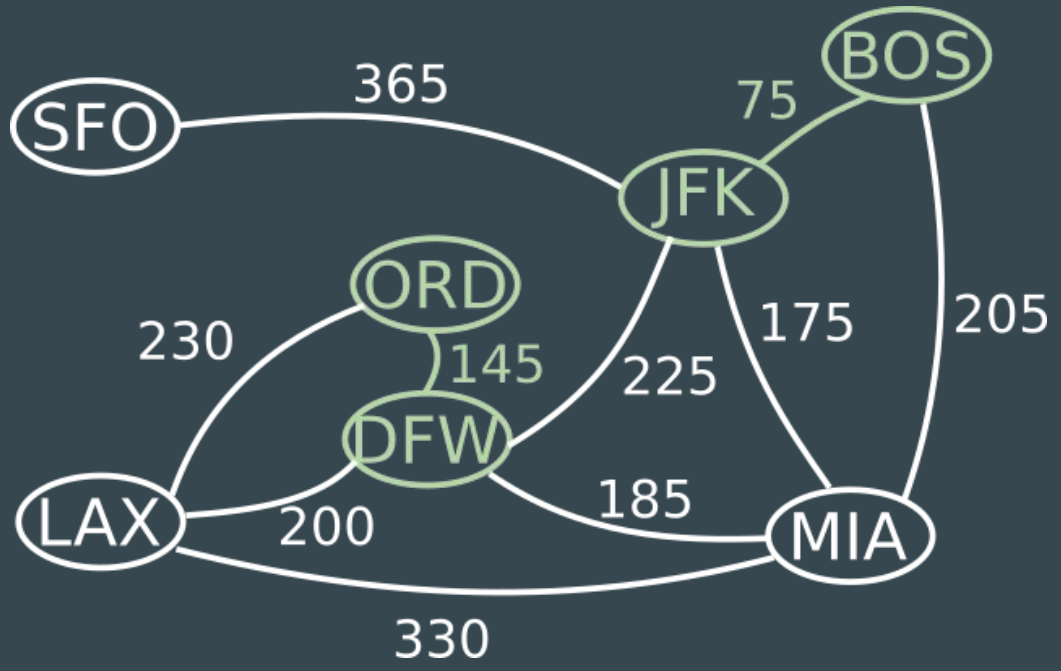
Algoritmo de Kruskal



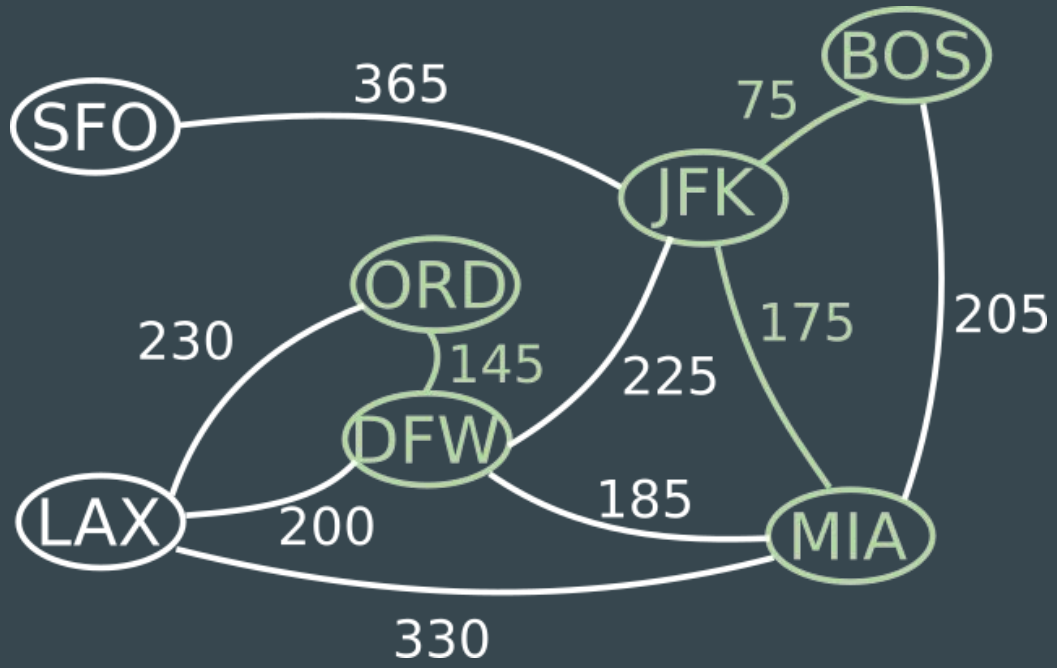
Algoritmo de Kruskal



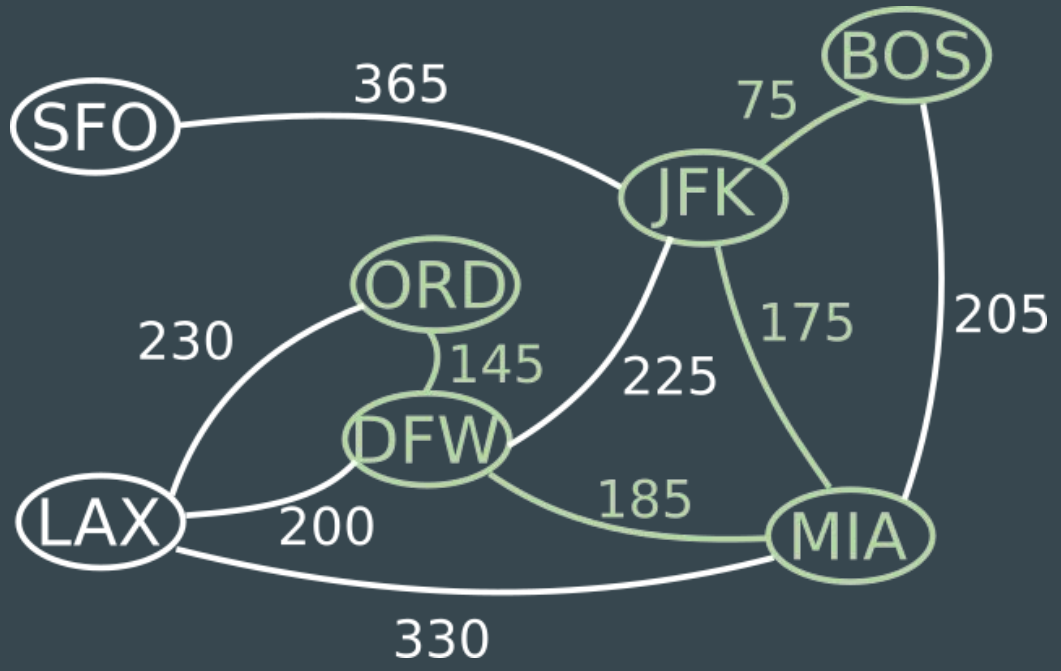
Algoritmo de Kruskal



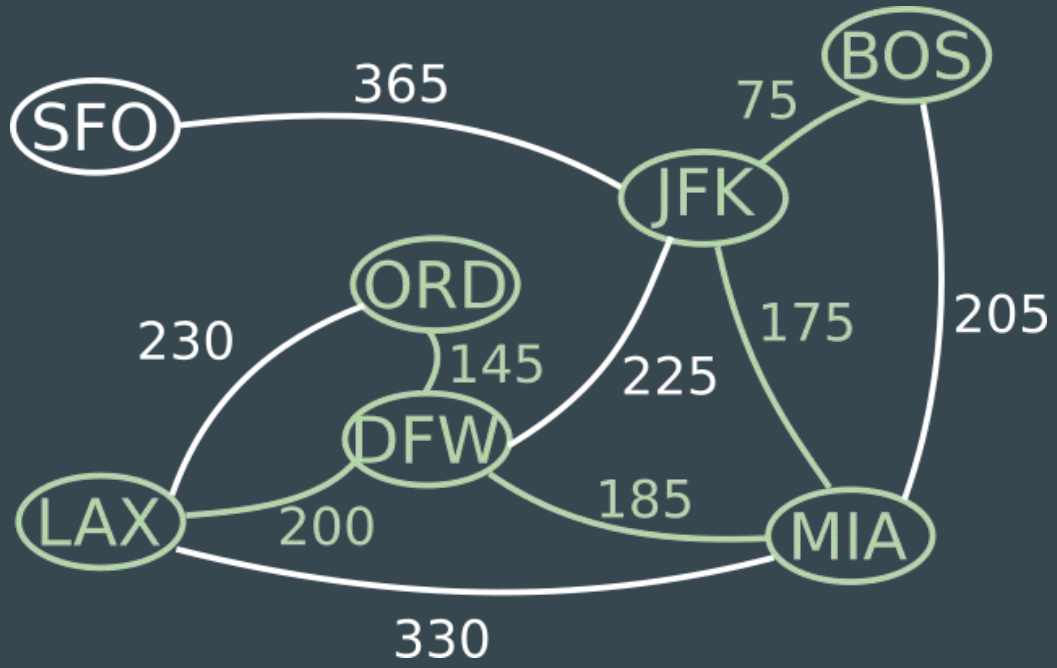
Algoritmo de Kruskal



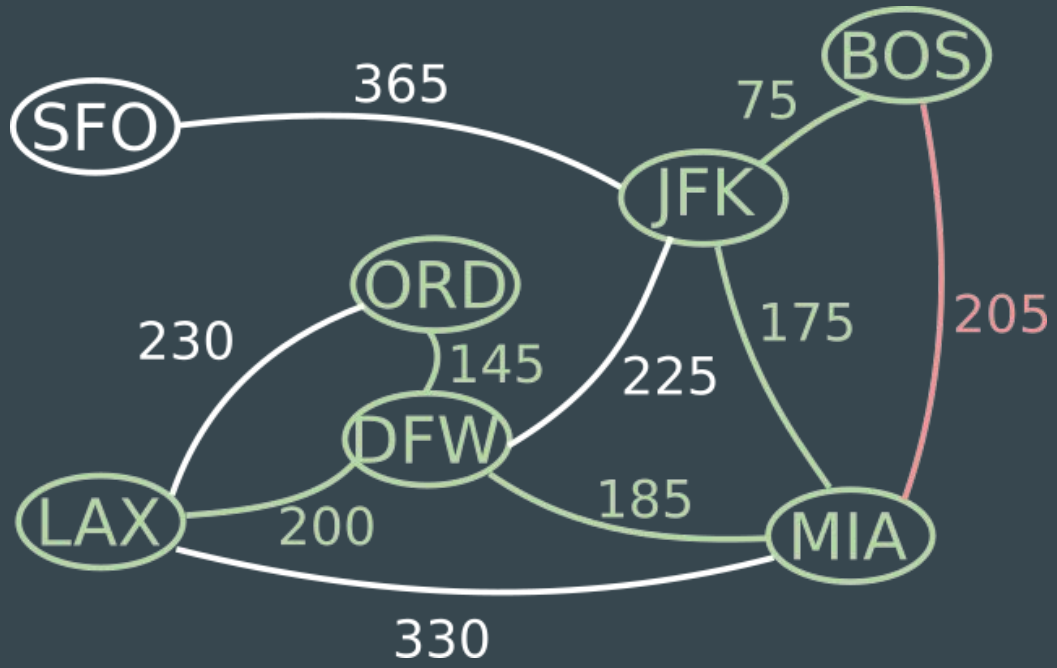
Algoritmo de Kruskal



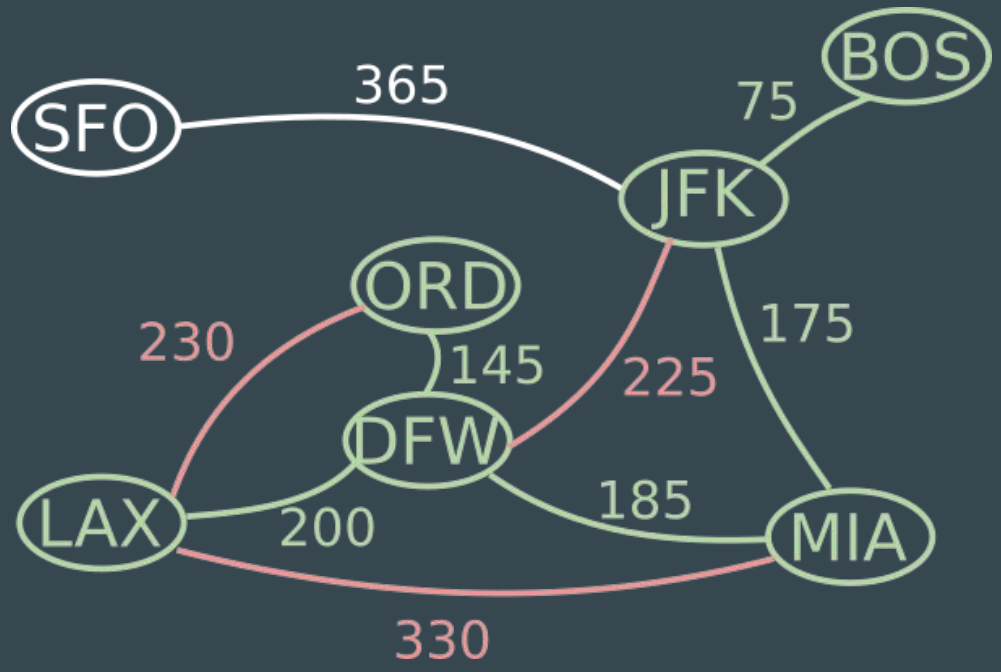
Algoritmo de Kruskal



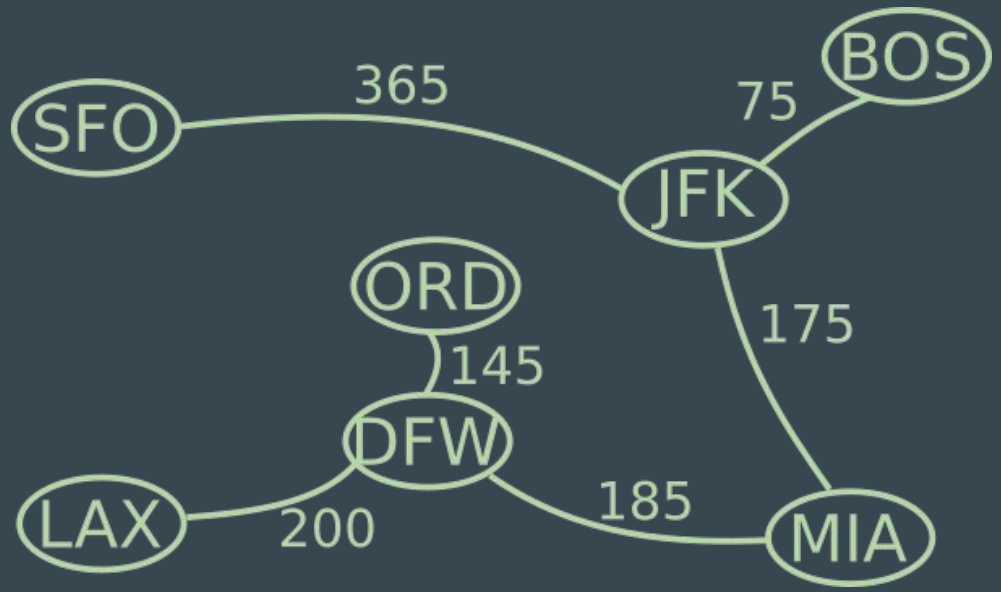
Algoritmo de Kruskal



Algoritmo de Kruskal



Algoritmo de Kruskal



Algoritmo de Kruskal: código

```
Entrada: grafo, no dirigido, ponderado

armar lista de subárboles (cada elemento será un subárbol)
armar tabla de pesos ordenada; cada arista tendrá el formato inicio, fin, peso
por cada candidato en tabla de pesos ordenada:
  ubicar en qué subárbol está cada nodo
  si inicio no están en ningún subárbol y fin no están en ningún subárbol:
    crear subárbol
  si inicio está en un subárbol y fin no está en ninguno:
    agregar fin al subárbol correspondiente
  si fin está en un subárbol e inicio no está en ninguno:
    agregar inicio al subárbol correspondiente
  si están en distintos subárboles:
    combinar subárboles
```

Algoritmo de Kruskal: código

```
def ordenar_grafo(grafo):
  tabla = [] # Se guardarán las aristas con el formato “inicio, fin, peso”
  aristas = 0 # contador del número de aristas

  for nodo in grafo:
    aristas += len(grafo[nodo]) # se suman todas las conexiones que tiene un nodo
  # Recorrer el grafo y llenarlo con las aristas ordenadas
  while len(tabla) <= aristas/2: # al ser no dirigido, cada arista está 2 veces
    menor = float("inf")
    for nodo1 in grafo: # cada nodo del grafo
      for nodo2 in grafo[nodo1]: # cada nodo conectado al nodo anterior
        if grafo[nodo1][nodo2] < menor and \
               (nodo1, nodo2, grafo[nodo1][nodo2]) not in tabla and \
               (nodo2, nodo1, grafo[nodo1][nodo2]) not in tabla:
          menor = grafo[nodo1][nodo2]
          candidato = (nodo1, nodo2, grafo[nodo1][nodo2])
    tabla.append(candidato)
  return tabla
```

Algoritmo de Kruskal: código

```
def kruskal(grafo):
  subarboles = [] # Cada subarbol que se vaya creando se guarda en un índice nuevo
  aristas_ordenadas = ordenar_grafo(grafo)

  for candidato in aristas_ordenadas:
    inicio = candidato[0]
    fin = candidato[1]
    peso = candidato[2]
    indice_inicio = -1
    indice_fin = -1

# Se busca en qué subarbol de la lista "subarboles" está cada nodo
# si no está, se guarda como -1
  for i in range(len(subarboles)):
    if inicio in subarboles[i]:
      indice_inicio = i
    if fin in subarboles[i]:
      indice_fin = i
```

Algoritmo de Kruskal: código

```
def ordenar_grafo(grafo):
  # continúa de diapositiva anterior

  # Si ningun nodo esta en un subarbol, se crea el subarbol con los dos nodos
  if indice_inicio == -1 and indice_fin == -1:
    arbol = {inicio: {fin: peso},
                fin: {inicio: peso} }
    subarboles.append(arbol)

  # Si el nodo de inicio no esta, se agrega al subarbol donde esta el fin
  elif indice_inicio == -1 and indice_fin != -1:
    subarboles[indice_fin][fin][inicio] = peso
    subarboles[indice_fin][inicio] = { fin: peso }

  # Si el nodo de fin no esta, se agrega al subarbol donde esta el inicio
  elif indice_inicio != -1 and indice_fin == -1:
    subarboles[indice_inicio][inicio][fin] = peso
    subarboles[indice_inicio][fin] = { inicio: peso }
```

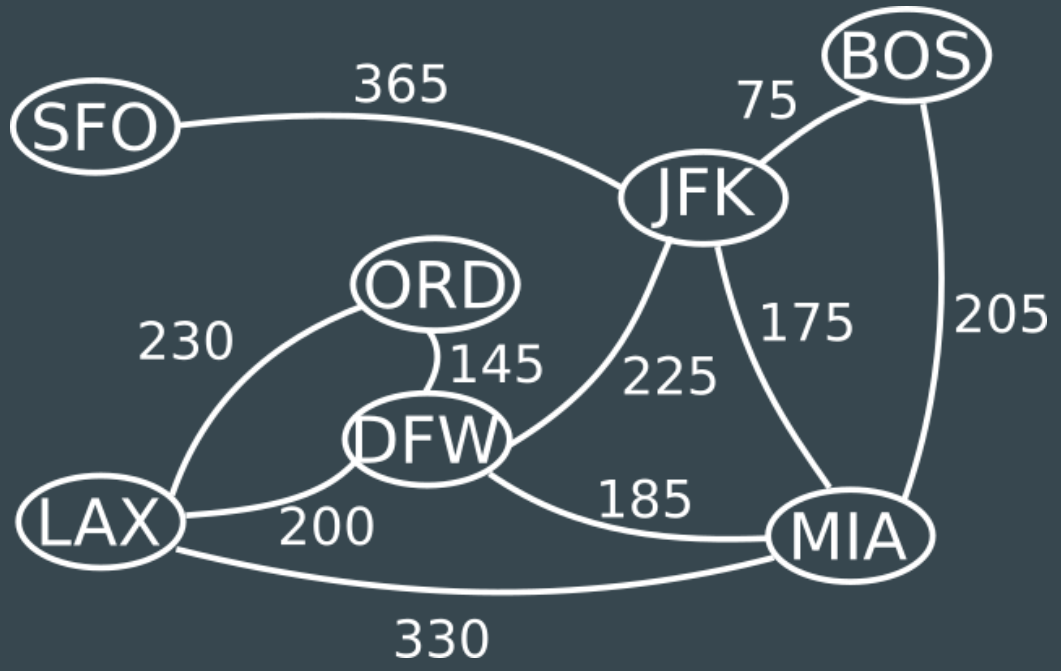
Algoritmo de Kruskal: código

```
def ordenar_grafo(grafo):
  # continúa de diapositiva anterior

  # Si los 2 estan pero en subarboles distintos, se combinan
  elif indice_inicio != indice_fin:
    for nodo in subarboles[indice_fin]:
      # Se copia la información de cada nodo del subárbol que tiene el nodo “fin”
      # en el subárbol que tiene al nodo “inicio”
      subarboles[indice_inicio][nodo] = subarboles[indice_fin][nodo]
    # Se agrega la arista entre el nodo “inicio” y el nodo “fin”
    subarboles[indice_inicio][inicio][fin] = peso
    subarboles[indice_inicio][fin][inicio] = peso
    subarboles.pop(indice_fin)

  return subarboles
```

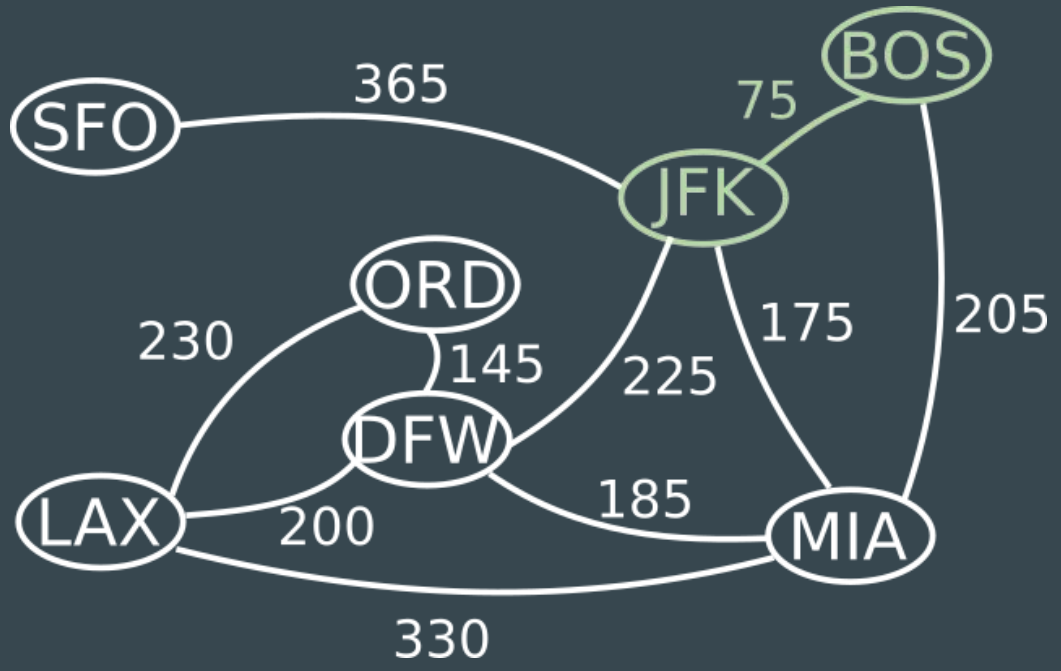
Algoritmo de Prim

- También es un algoritmo ávido.
- Se comienza eligiendo la **arista de menor peso**, y se continúa con la arista de menor peso conectada a los nodos **ya visitados**.
- Si la arista de menor peso lleva a un nodo ya visitado, se descarta.
- El algoritmo termina cuando todos los nodos fueron visitados

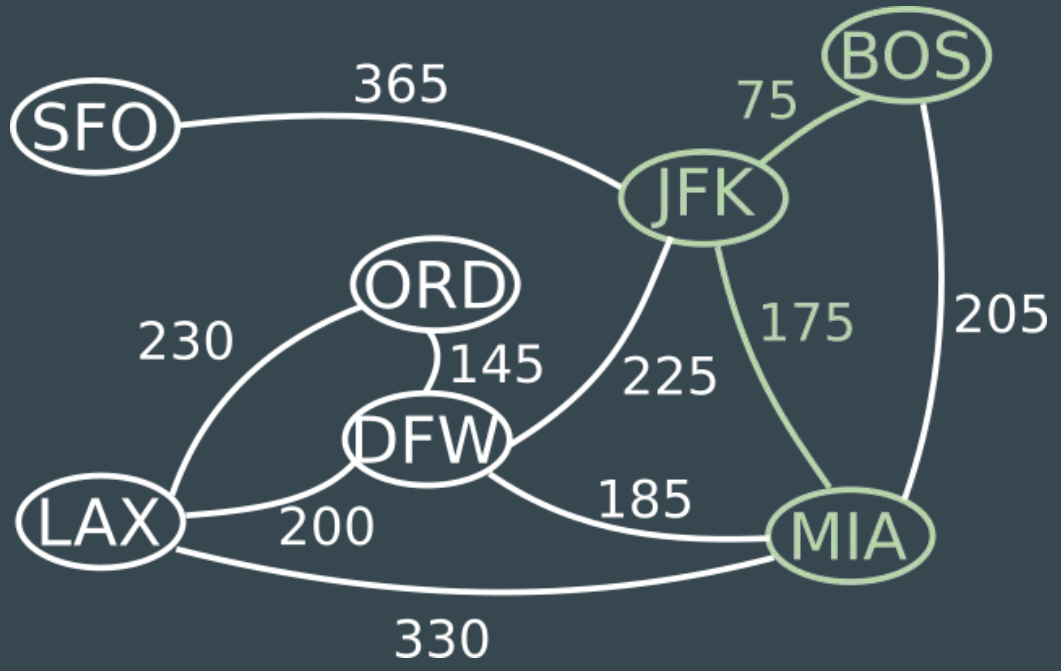
Algoritmo de Prim



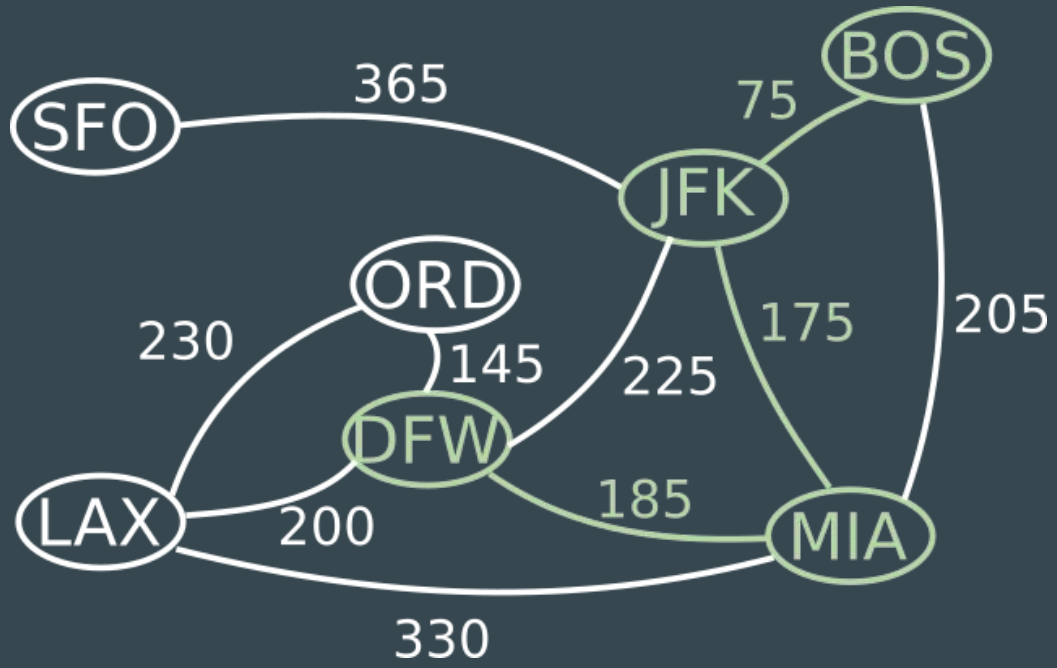
Algoritmo de Prim



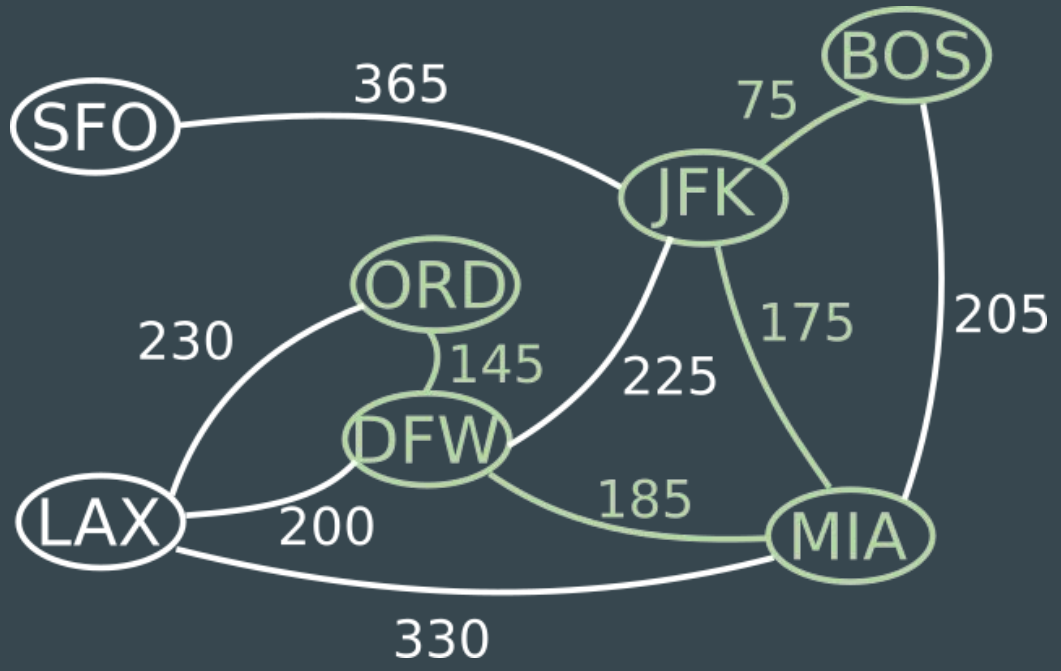
Algoritmo de Prim



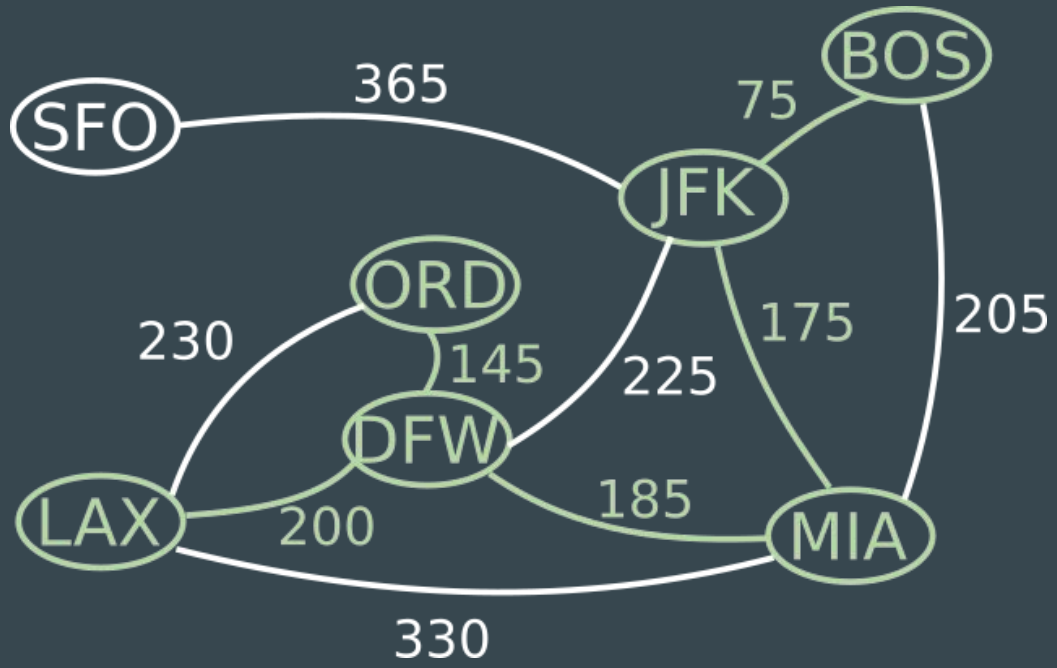
Algoritmo de Prim



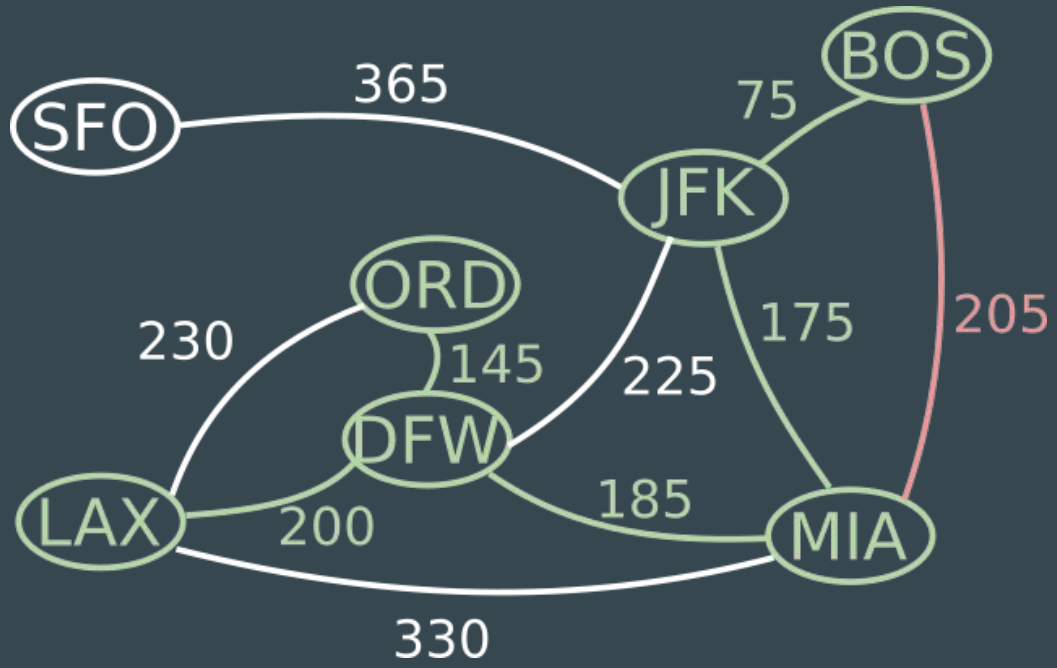
Algoritmo de Prim



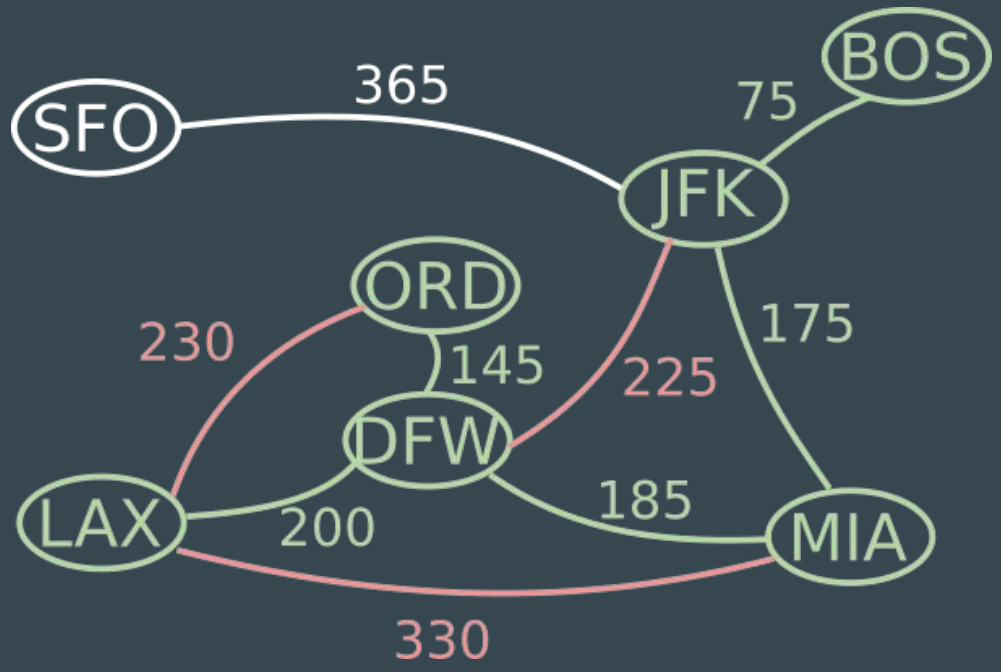
Algoritmo de Prim



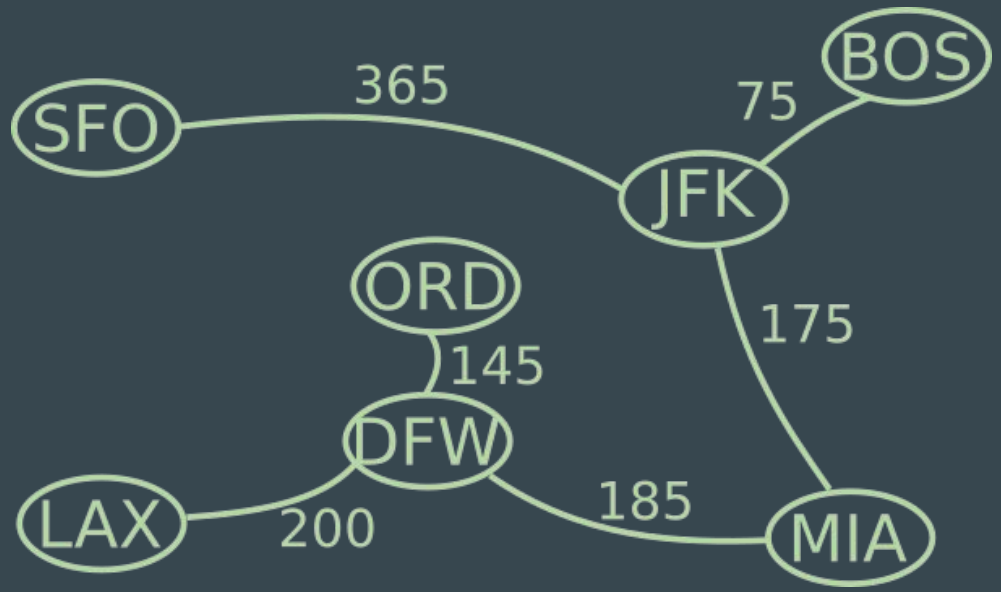
Algoritmo de Prim



Algoritmo de Prim



Algoritmo de Prim



Algoritmo de Prim: pseudocódigo

```
Entrada: grafo, no dirigido, ponderado

mientras len(arm) < len(grafo):
  seleccionar arista de menor peso conectada a nodos en arm
  si nodo_destino no está en arm:
    añadir arista al arm
```

Algoritmo de Prim: código

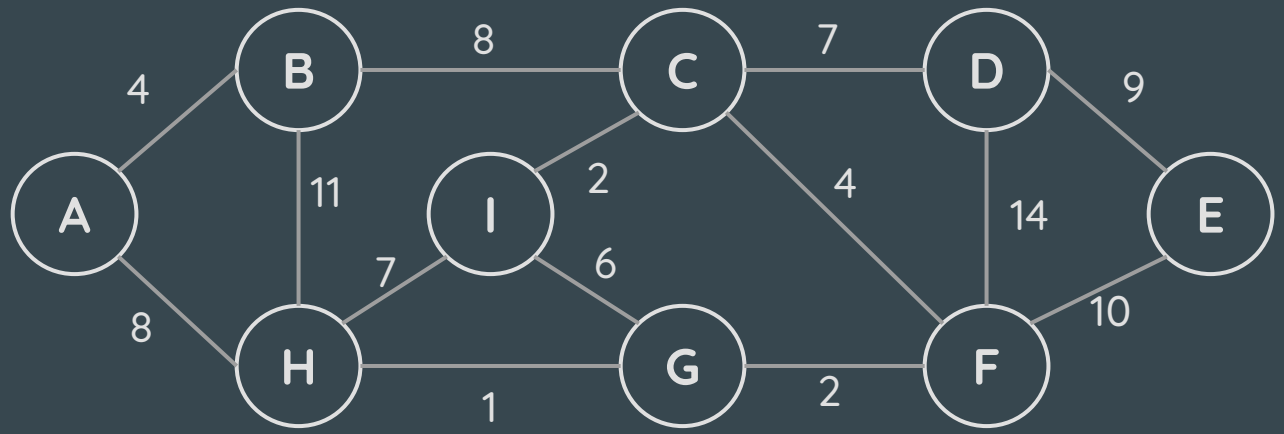
```
def prim(grafo):
  arbol = {}
  while len(arbol) < len(grafo):
    inicio, fin, peso = elegir_candidatos(grafo, arbol)
    if inicio not in arbol:
      arbol[inicio] = {}
    if fin not in arbol:
      arbol[fin] = {}
    arbol[inicio][fin] = peso
    arbol[fin][inicio] = peso
  return arbol
```

Algoritmo de Prim: código

```
def elegir_candidatos(grafo, arm):
  menor = float("inf")
  if len(arm) == 0: # primer iteración, no hay nada en el árbol
    for nodo1 in grafo:
      for nodo2 in grafo[nodo1]:
        if grafo[nodo1][nodo2] < menor:
          menor = grafo[nodo1][nodo2]
          inicio = nodo1
          fin = nodo2
  else: # si el árbol tiene algún nodo, se busca ahí
    for nodo1 in arm:
      for nodo2 in grafo[nodo1]:
        if grafo[nodo1][nodo2] < menor and nodo2 not in arm:
          menor = grafo[nodo1][nodo2]
          inicio = nodo1
          fin = nodo2
  return inicio, fin, menor
```

Ejemplo

A partir del siguiente grafo, calcule los árboles recubridores mínimos usando los algoritmos de Kruskal y de Prim



Bibliografía

- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data structures and algorithms in Python. Capítulo 14.7
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms. MIT press. Capítulo 23